MODULE 02 · DOMAIN 1 - ARCHITECTURE BUILT FOR THE CLOUD

Stateless Design and Durable Workflows

Where the state lives decides how you scale, how you recover, and what you pay for while you wait.

DOMAIN 1 of 5 EXAM WEIGHT 25% MODULE 2 of 52

admin@sthithapragna.com

A server you can replace at any moment must **remember nothing**. The state has to live somewhere else.

Where each kind of state belongs

STATE	PUT IT IN	WHY
User sessions	ElastiCache or DynamoDB	Shared, expires with TTL
Uploaded files	Amazon S3	Not the instance disk
Workflow progress	Step Functions or durable functions	Survives failures and waits
WebSocket connection IDs	DynamoDB	Any backend can push
Config and secrets	Parameter Store, Secrets Manager	Never baked into images

ON THE EXAM

Compute holds nothing it cannot rebuild from a shared store.

Sticky sessions or externalised state?

STICKY SESSIONS

State on the instance

- Load balancer pins a user to one target
- No code change needed
- Session lost when that target goes
- Uneven load as users accumulate

EXTERNALISED STATE

State in a shared store

- Any target can serve any request
- Scale in and deploy without loss
- Small code change to read and write sessions
- Even load across the fleet

THE TIEBREAKER

Stickiness hides statefulness. It does not remove it, so scale-in and Spot still lose sessions.

Session store: ElastiCache or DynamoDB?

CHOOSE ELASTICACHE WHEN

Lowest latency, one Region

- Sub-millisecond reads at very high rates
- Valkey or Redis OSS, with key TTL
- Serverless option removes node sizing
- Global Datastore: secondaries are read-only

CHOOSE DYNAMODB WHEN

Durable and multi-Region

- Single-digit millisecond, fully managed
- TTL deletes expired sessions
- Global tables accept writes in every Region
- No cluster to size or patch

THE TIEBREAKER

Writes from more than one Region rule out Global Datastore. That is a DynamoDB global tables question.

A workflow that waits for days should **not hold compute**, and should not lose its place when something fails.

Three ways to run a multi-step workflow

OPTION	DURATION	SEMANTICS	BILLED BY
Step Functions Standard	Up to 1 year	Exactly once	State transitions
Express, asynchronous	Up to 5 minutes	At least once	Executions, duration, memory
Express, synchronous	Up to 5 minutes	At most once	Executions, duration, memory
Lambda durable functions	Up to 1 year	Checkpoint and replay	Invocations; waits not charged

ON THE EXAM

The semantics column decides more questions than the duration column. Non-idempotent steps need exactly once.

Step Functions: Standard or Express?

CHOOSE STANDARD WHEN

Long, auditable, non-idempotent

- Runs longer than five minutes
- Steps must not repeat, such as payments
- Waits for a callback or a .sync job
- Needs 90 days of execution history

CHOOSE EXPRESS WHEN

Short, high-volume, idempotent

- Finishes inside five minutes
- Thousands of executions per second
- Steps are safe to repeat
- Cost per execution matters most

THE TIEBREAKER

Callbacks and .sync jobs exist only in Standard. Waiting on anything external rules Express out.

Step Functions or Lambda durable functions?

CHOOSE STEP FUNCTIONS WHEN

Orchestrate services

- Direct integrations with many AWS services
- Visual workflow and execution history
- Distributed Map for large parallel batches
- Logic defined in Amazon States Language

CHOOSE DURABLE FUNCTIONS WHEN

Workflow as application code

- Steps and waits written in the function's code
- Python, Node.js, Java, Go or .NET SDK
- Waits pause execution and compute charges
- Team stays inside Lambda

THE TIEBREAKER

Both are durable for up to a year. The deciding detail is where the team wants the workflow logic to live.

Durable functions: the rules that bite

- **Replay from the start.** Lambda reruns your code and substitutes stored results for completed steps.
- **Code must be deterministic.** No random values or clock reads outside a step.
- **Side effects go inside steps,** so a completed step never runs again.
- **Execution timeout** defaults to 24 hours, up to 1 year. History kept 1 to 90 days.
- **Cross-account invoke** of another function is not supported.

> ON THE EXAM
>
> Checkpoint and replay means anything outside a step runs again on every resume. Keep it pure.

Waiting for people and partner systems

- **Task token callback:** Standard workflow pauses until SendTaskSuccess or SendTaskFailure.
- **Durable function waits** pause on a timer or an external callback.
- **Heartbeat and timeout** on the waiting state catch callers that never reply.
- **Polling loops in Lambda** pay for idle time and stop at 15 minutes.
- **Skill 2.5.3** reuses this pattern for human approval of AI actions.

> ON THE EXAM
>
> A wait measured in hours or days is a callback, never a loop. Idle compute is the wrong answer.

Real-time push with API Gateway WebSocket APIs

01

Connect

$connect route stores the connection ID in DynamoDB.

02

Subscribe

A message route records what the client follows.

03

Event

A backend change is published to EventBridge or SNS.

04

Push

Lambda posts to each connection via @connections.

05

Disconnect

$disconnect route deletes the stale ID.

ON THE EXAM

API Gateway holds the socket. Your backend only needs the connection ID and the @connections endpoint.

WebSocket limits that shape the design

- **Connection duration: 2 hours.** Not adjustable. Clients must reconnect.
- **Idle timeout: 10 minutes.** Not adjustable. Send a heartbeat.
- **Message payload: 128 KB,** sent in frames of 32 KB or less.
- **500 new connections per second** per account per Region. This one is adjustable.
- **Push, do not poll.** Pushing a change beats thousands of clients asking every second.

> ON THE EXAM
>
> A quota increase is never the answer to a 2-hour or 10-minute disconnect. Those limits are fixed.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
Logged out during scale-in or deploy	Session state on the instance
Users write in either Region	DynamoDB global tables, not Global Datastore
Waits days for a person or partner	Standard callback or a durable wait
Thousands per second, idempotent steps	Express workflows on cost
Team wants the workflow in its own code	Lambda durable functions
Disconnects at 2 hours or 10 minutes	Fixed WebSocket limits; fix the client

ON THE EXAM

Each phrase is a constraint, and each constraint removes at least two options.

> Meridian Freight's booking site runs on EC2 instances in an Auto Scaling group behind an Application Load Balancer with sticky sessions. Shopping carts are held in instance memory.
>
> After scale-in events and deployments, customers report empty carts and being logged out. The company wants to keep scaling aggressively.

Which change prevents the lost sessions?

A Increase the target group's deregistration delay to 30 minutes

B Switch to a Network Load Balancer with source IP affinity

C Store sessions in Amazon ElastiCache and disable stickiness

D Use larger instances so scaling happens less often

◆ PAUSE AND DECIDE

Which change prevents the lost sessions?

A Longer deregistration delay

Lets in-flight requests finish. The instance still terminates and its memory goes with it.

B NLB with source IP affinity

Another form of stickiness. Same loss when the target goes, and worse behind shared NAT.

C Sessions in ElastiCache

Any instance can serve any user, so termination loses nothing.

D Larger instances

Fewer losses, not none, and it contradicts the wish to scale aggressively.

THE SIGNAL

Loss on scale-in and deploy means state on the instance. Every answer that keeps it there only delays the loss.

> Meridian Freight runs its customer portal active-active in eu-west-1 and us-east-1. Route 53 latency routing can send a user to either Region on any request, and users update their session data as they work.
>
> Session changes must be visible in the other Region within about a second, and sessions must survive a node failure.

Which session store meets these requirements?

A. A separate ElastiCache cluster in each Region

B. A DynamoDB global table with TTL on the session item

C. An ElastiCache Global Datastore spanning both Regions

D. An S3 bucket in each Region with Cross-Region Replication

◆ PAUSE AND DECIDE

Which session store meets these requirements?

A. Separate clusters per Region

Nothing replicates. A user routed to the other Region starts a new session.

B. DynamoDB global table

Writes accepted in both Regions, replicated typically within a second, durable.

C. ElastiCache Global Datastore

Secondary clusters only accept reads. Session writes in the second Region fail.

D. S3 with replication

Replication is asynchronous and can take minutes. Not a session store.

THE SIGNAL

Users update sessions in either Region. A read-only secondary cannot accept those writes.

> Meridian Freight is automating damage claims. The workflow validates the claim, waits for an independent surveyor's report, then pays the customer through a payments API that is not idempotent.
>
> Surveyor reports can take up to three weeks. Auditors need the history of each claim's execution.

Which approach should the solutions architect use?

A. An asynchronous Express workflow with a Wait state

B. A Lambda function that polls for the report every hour

C. A synchronous Express workflow invoked from API Gateway

D. A Standard workflow using a task token callback for the surveyor's report

◆ PAUSE AND DECIDE

Which approach should the solutions architect use?

A. Async Express with Wait

Five-minute limit, and at-least-once semantics could repeat the payment.

B. Polling Lambda

Pays for repeated invocations, and holds workflow state nowhere durable.

C. Sync Express

Five-minute limit, and no execution history for auditors.

D. Standard with callback

Waits weeks without charge, runs the payment exactly once, keeps history.

THE SIGNAL

Three weeks rules out Express. A non-idempotent payment and an audit history both point at Standard.

> Meridian Freight ingests about 3,000 scanner events per second. Each event passes through four short transformation steps, about 200 milliseconds in total, and ends with a DynamoDB PutItem keyed on the scan ID.
>
> The current Standard workflow costs far more than expected.

Which change is MOST cost-effective?

A Change to an asynchronous Express workflow

B Keep Standard and remove the Retry blocks to reduce transitions

C Batch events and process them with a Distributed Map in Standard

D Convert each step to a separate Standard workflow

◆ PAUSE AND DECIDE

Which change is MOST cost-effective?

A Asynchronous Express

Billed by execution and duration, and the PutItem on a fixed key is safe to repeat.

B Remove retries

Retries only add transitions when they fire. The four steps are still billed every time.

C Distributed Map in Standard

Built for large batch jobs, and it adds buffering latency to a stream.

D One workflow per step

Multiplies executions and transitions. Strictly more expensive.

THE SIGNAL

High volume, short duration, idempotent steps. Every one of those words describes Express.

A Meridian Freight team owns a customer onboarding process written in Python and deployed as Lambda functions. It must pause for up to seven days while a customer uploads verification documents, then continue.

The team wants the whole workflow in its existing codebase and test suite, does not want to learn Amazon States Language, and must not pay for compute while waiting.

Which solution meets these requirements with the LEAST change?

A. Rewrite the process as a Step Functions Standard workflow with a callback

B. Re-enqueue a message on an SQS delay queue until the documents arrive

C. A Lambda durable function, with steps for each stage and a wait for the upload callback

D. An EventBridge Scheduler schedule that invokes the function daily to check progress

◆ PAUSE AND DECIDE

Which solution meets these requirements with the LEAST change?

A **Step Functions rewrite**

Technically sound, but it moves the logic into States Language, which the team ruled out.

B **SQS delay loop**

Delay tops out at 15 minutes, so it becomes hundreds of re-enqueues and hand-built state.

C **Lambda durable function**

Workflow stays in Python, checkpoints each step, and waits without compute charges.

D **Daily scheduled check**

Workflow state must be stored and reloaded by hand, and progress lags by up to a day.

THE SIGNAL

The constraint is not technical. The team wants the workflow in its own code, which is what durable functions add.

> Meridian Freight's live tracking page receives shipment positions through an API Gateway WebSocket API. Dispatchers keep the page open all day.
>
> Dispatchers report that the page stops updating roughly every two hours, and that quiet shipments stop updating after about ten minutes without a position change.

What should the solutions architect do?

A Request a quota increase for WebSocket connection duration and idle timeout

B Have the client send periodic heartbeat messages and reconnect automatically when closed

C Increase the timeout of the Lambda function behind the $default route

D Replace the WebSocket API with a REST API that the page polls every second

◆ PAUSE AND DECIDE

What should the solutions architect do?

A Quota increase

Neither limit is adjustable. The request cannot be granted.

B Heartbeat and auto-reconnect

Keeps quiet connections alive and recovers cleanly from the 2-hour close.

C Longer Lambda timeout

The function is not holding the connection. API Gateway is.

D REST polling every second

Thousands of mostly empty requests, more cost and load for a worse experience.

THE SIGNAL

Two hours and ten minutes are the fixed WebSocket limits. Round-number failures are usually quotas.

What to carry out of this module

- **Lost sessions on scale-in** mean state on the instance. Move it out, do not protect the instance.
- **Writes in two Regions:** DynamoDB global tables. Global Datastore secondaries are read-only.
- **Standard** for long, exactly-once, callback. **Express** for short, high-volume, idempotent.
- **Durable functions** since December 2025: durable workflows written as Lambda code.
- **WebSocket 2-hour and 10-minute limits** are fixed. Heartbeat and reconnect.

> ON THE EXAM
>
> Older courses have no durable functions. When code-first workflows appear, that option is now a real answer.